

Reviewer Pack — Hypergeometric Function Moments Bound ACC Circuit Size

Entry d6e65f983582 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 12:16:35 UTC

Hypergeometric Function Moments Bound ACC Circuit Size

Entry ID: d6e65f983582 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 12:16:35 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Functions **Field B** (complexity object): ACC Circuit Size

Statement:

For any instance of an ACC circuit with n variables, the sum of moments of its characteristic polynomial over a specified field F is upper-bounded by $O(n \log n)$. Equivalently, for every CNF formula with n variables, if F is the field of two elements, then the hypergeometric function moments of its defining polynomials are upper-bounded by $O(\log n)$ over the complex numbers.

Rationale (proposer's reasoning):

Hypergeometric functions provide a rich algebraic structure that could potentially expose hidden properties in ACC circuits. If these moments can be controlled by circuit size, it might suggest new techniques for proving lower bounds on ACC complexity.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bacd0145be6a3215

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an ACC circuit with n variables, if the sum of moments of its characteristic polynomial over the complex numbers is $\leq 0.01 * n \log(n)$ and for CNF formulas, the hypergeometric function moments are $\leq 0.01 * \log(n)$, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hypergeometric Functions" AND "ACC circuit size" - "characteristic polynomial" AND "hypergeometric functions" AND "circuit complexity" - "CNF formula" AND "hypergeometric function moments" AND "complex numbers"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def matrix_multiply(A, B):
    m = len(A)
    k = len(B[0])
    result = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(len(B)):
                result[i][j] += A[i][l] * B[l][j]
    return result
```

```

def characteristic_polynomial(acc_circuit):
    n = len(acc_circuit)
    x = Fraction('x')
    poly = [Fraction(1)]
    for i in range(n):
        new_poly = []
        for term in poly:
            new_poly.append(term * x - acc_circuit[i])
        poly = new_poly
    return poly

def hypergeometric_moments(poly, n):
    moments = [0] * (n + 1)
    for i in range(n + 1):
        moment = Fraction(1)
        for j in range(i):
            moment *= poly[j]
        moments[i] = moment
    return sum(moments)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    acc_circuit = [random.choice([-1, 1]) for _ in range(n)]
    characteristic_poly = characteristic_polynomial(acc_circuit)
    moments_sum = hypergeometric_moments(characteristic_poly, n)

    metric_name = "moments_sum"
    metric_value = abs(moments_sum) * n * math.log(n)
    instances_tested = 1
    conjecture_holds = metric_value <= 0.01 * n * math.log(n)
    counterexample = "" if conjecture_holds else f"n={n}, moments_sum={moments_sum}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73]
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04c44f9b.py", line 98, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04c44f9b.py", line 71, in run_trial
    characteristic_poly = characteristic_polynomial(acc_circuit)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04c44f9b.py", line 49, in characteristic_polynomial
    x = Fraction('x')
        ^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 239, in __new__
    raise ValueError('Invalid literal for Fraction: %r' %
ValueError: Invalid literal for Fraction: 'x'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15983
2	preregistration	ollama_remote	glm4:latest	0	0	10466
3	novelty	ollama_remote	glm4:latest	0	0	9028
4	novelty	ollama_remote	glm4:latest	0	0	10195
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17101
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10294
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15599
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12619
9	judge	ollama_remote	glm4:latest	0	0	12099

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113385 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d6e65f983582.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d6e65f983582.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d6e65f983582.tar.gz` (if generated)